Running Cast-History Analysis on a C Project

This document describes the recommended workflow for running the cast-history tool on a project, validating the output, and exporting tables for experiments or paper results. The workflow assumes the C6f line of the tool, including stable classification output, JSON schema validation, compile-database-based project running, and collision-safe output naming.

1. Expected Project Structure

Each analyzed project should provide a Clang-compatible compilation database:

```
project-root/
  compile_commands.json
  src/
    ...
  include/
    ...
```

The runner can also be pointed directly at a build directory or directly at a `compile_commands.json` file. The important requirement is that every translation unit to be analyzed appears in `compile_commands.json` with the correct compiler flags, include paths, macro definitions, and working directory.

The runner analyzes source files listed in the compilation database. Header files are normally analyzed indirectly when included by a listed translation unit. A header is not analyzed as a standalone translation unit unless it appears in the compilation database or is explicitly treated as a source.

For larger projects, source filters can be used to include or exclude subsets of the project, for example to exclude tests, generated files, vendored dependencies, or third-party code.

Recommended real-project layout:

```
project-root/
  compile_commands.json
  src/
  include/
  lib/
  tests/                # optional, often excluded for paper runs
  third_party/          # often excluded
```

Recommended output layout:

```
test-output/
  real-projects/
    project-name/
      commands.log
      project-summary-index.json
      project-summary-index-used-for-final-import.json
      import-round-01-index-input.json
      import-round-02-index-input.json
      *.txt
      summaries/
        *.cast-summary.json
```

The `.txt` files are the human-readable final import-aware reports. The files under `summaries/` are the final import-aware JSON summaries and should be treated as the primary machine-readable output.

2. Running One Project

From the build directory:

```
cd build

PROJECT_PATH=/path/to/project-or-build-dir
PROJECT_NAME=my_project
OUTPUT_ROOT=../test-output/real-projects
SUMMARY_IMPORT_ROUNDS=2
../scripts/run_all_multitu_projects.sh
```

Alternatively, point directly to the compilation database:

```
cd build

PROJECT_PATH=/path/to/compile_commands.json
PROJECT_NAME=my_project
OUTPUT_ROOT=../test-output/real-projects
SUMMARY_IMPORT_ROUNDS=2
../scripts/run_all_multitu_projects.sh
```

Useful filters for larger projects:

```
SOURCE_FILTER_REGEX='/(src|lib)/'
SOURCE_EXCLUDE_REGEX='/(test|tests|third_party|vendor|build)/'
PROJECT_PATH=/path/to/project-or-build-dir
PROJECT_NAME=my_project
```

```
OUTPUT_ROOT=../test-output/real-projects
SUMMARY_IMPORT_ROUNDS=2
../scripts/run_all_multitu_projects.sh
```

For reproducible pilot runs, record the exact command line, the git commit of the analyzed project, the git commit of the cast-history tool, and the generated `commands.log`.

3. Ordering of Analysis

The runner performs the analysis in several phases.

Phase C1: raw per-translation-unit summaries

Each source file from `compile_commands.json` is analyzed once without summary import. This produces raw per-TU summaries.

Conceptually:

```
source file -> raw cast-summary JSON
```

These summaries are not yet fully import-aware, but they are needed to build the first project-level summary index.

Phase C2: build the project summary index

The script then invokes:

```
scripts/build_summary_index.py
```

on the raw summaries. This creates:

```
project-summary-index.json
```

The index records the summaries available for each translation unit and enables cross-TU import, function-summary lookup, function-pointer target lookup, SCC-aware summary solving, and other project-level composition.

Import refinement rounds

The runner then performs one or more import-aware rounds.

Each import round does this:

```
previous project-summary-index.json
  -> analyze each translation unit with --enable-summary-import
  -> write import-aware summaries
  -> rebuild project-summary-index.json from those summaries
```

With `SUMMARY_IMPORT_ROUNDS=2`, the conceptual workflow is:

```
C1 raw summaries
  -> index0
  -> import round 1 summaries
  -> index1
  -> import round 2 summaries
  -> final index
```

Use more rounds when information must propagate through longer dependency chains, recursive SCCs, or several layers of callers. For small test projects, one or two rounds are usually enough. For real projects, two rounds are a reasonable default.

Final publishing step

After the last import round, the runner publishes the latest outputs to:

```
<OUTPUT_ROOT>/<PROJECT_NAME>/
```

The final text reports are stored as:

```
<OUTPUT_ROOT>/<PROJECT_NAME>/*.txt
```

The final JSON summaries are stored as:

```
<OUTPUT_ROOT>/<PROJECT_NAME>/summaries/*.cast-summary.json
```

The current runner uses collision-safe names based on relative source paths, so two files such as:

```
src/a.c
tests/a.c
```

will be written as distinct output files, for example:

```
src_a.c.txt
tests_a.c.txt

summaries/src_a.c.cast-summary.json
summaries/tests_a.c.cast-summary.json
```

4. Important Output Files

For each project, the most important outputs are:

```
commands.log
```

Records the commands used for each analysis step. This should be kept for reproducibility.

```
*.txt
```

Human-readable final reports for each translation unit. These are useful for debugging and manual inspection.

```
summaries/*.cast-summary.json
```

Final import-aware JSON summaries. These are the primary files for downstream analysis and paper tables.

```
project-summary-index.json
```

The final index rebuilt from the final import-aware summaries.

```
project-summary-index-used-for-final-import.json
```

The index consumed by the final import round. This helps reproduce the final import-aware analysis.

```
import-round-XX-index-input.json
```

The index consumed by each import round. These files are useful when debugging why a certain summary was or was not imported.

5. Validation After Running a Project

After a project completes, run the JSON classification-label validator:

```
python3 ../scripts/validate_classification_labels.py
  --root ../test-output/real-projects/my_project
```

This default mode checks for semantic contradictions and obvious schema problems while remaining compatible with older or partially populated metadata.

For stricter checking:

```
python3 ../scripts/validate_classification_labels.py
  --strict-vocabulary
  --root ../test-output/real-projects/my_project
```

Strict mode is useful when freezing a freshly regenerated corpus. However, strict mode may expose metadata-completeness issues that are not semantic correctness bugs. For example, a composite mediation kind may be correctly classified but may lack a normalized `fields` list in some summaries. Such cases should be documented as schema-completeness debt unless they affect the paper result being extracted.

Next, validate the JSON summaries against the schema:

```
python3 ../scripts/validate_cast_summary_schema.py
  --root ../test-output/real-projects/my_project
```

If the optional `jsonschema` package is unavailable, use structural validation:

```
python3 ../scripts/validate_cast_summary_schema.py
  --no-jsonschema
  --root ../test-output/real-projects/my_project
```

For regression testing, it is also useful to validate consistency between text and JSON outputs if the project belongs to the curated test suite:

```
python3 ../scripts/validate_cast_text_json_consistency.py
  --root ../test-output/multitu
```

For large real projects, text/JSON consistency validation may need filtering or sampling because text output is intended primarily for human inspection.

6. Exporting Tables for Paper Results

The primary extraction script is:

```
scripts/extract_stable_classifications.py
```

Run it as:

```
python3 ../scripts/extract_stable_classifications.py
  --root ../test-output/real-projects/my_project
  --out ../test-output/real-projects/my_project/stable-classifications.csv
```

For multiple projects:

```
python3 scripts/extract_stable_classifications.py
  --root test-output/real-projects
  --out results/stable-classifications-all-projects.csv
```

The CSV contains one row per stable classification record, including both entity-level and parameter-level rows. Typical columns include:

```
summaryFile
sourceFile
subjectKind
subject
behavior
mediation
family
risk
evidenceSourceKind
fields
callbacks
targets
edges
flag_dataField
flag_nestedDataField
flag_callback
flag_externalBoundary
flag_returnAlias
```

Recommended paper tables include:

Table 1: Semantic family distribution

Count how many stable classifications fall into each family:

```
import pandas as pd

df = pd.read_csv("results/stable-classifications-all-projects.csv")

table = (
    df.groupby(["family"])
      .size()
      .reset_index(name="count")
      .sort_values("count", ascending=False)
)

table.to_csv("results/table-family-distribution.csv", index=False)
```

Table 2: Mediation kind distribution

```
table = (
    df.groupby(["mediation"])
      .size()
      .reset_index(name="count")
      .sort_values("count", ascending=False)
)

table.to_csv("results/table-mediation-distribution.csv", index=False)
```

Table 3: Risk by semantic family

```
table = (
    df.groupby(["family", "risk"])
      .size()
      .reset_index(name="count")
      .sort_values(["family", "risk"])
)

table.to_csv("results/table-family-by-risk.csv", index=False)
```

Table 4: Surface-wild refinement

For the paper's main argument, focus on rows whose surface kind or behavior indicates a `void *`-like interface, then show how many are actually structured:

```
structured = df[df["family"].isin([
    "DirectConcreteSpecialization",
    "DataFieldMediatedConcreteSpecialization",
    "NestedDataFieldMediatedConcreteSpecialization",
    "CallbackMediatedConcreteSpecialization",
    "GuardedCallbackMediatedSpecialization",
    "ReturnAliasMediatedConcreteSpecialization",
    "ByteView",
    "OpaqueTransport",
])]

table = (
    structured.groupby(["family"])
              .size()
              .reset_index(name="count")
              .sort_values("count", ascending=False)
)

table.to_csv("results/table-surface-wild-refinement.csv", index=False)
```

Table 5: Project-level distribution

If the extractor includes project names or summary paths, derive the project from `summaryFile`:

```
from pathlib import Path

df["project"] = df["summaryFile"].apply(lambda p: Path(p).parts[-3] if
len(Path(p).parts) >= 3 else "")

table = (
    df.groupby(["project", "family"])
      .size()
      .reset_index(name="count")
      .sort_values(["project", "count"], ascending=[True, False])
)

table.to_csv("results/table-project-family-distribution.csv", index=False)
```

These tables can support claims such as:

```
Many surface-wild void* interfaces are not semantically wild.
They are often direct concrete specializations, data-field-mediated
```

```
specializations,
callback-mediated specializations, byte views, or opaque transports.
```

7. Purpose and Usage of Each Script

`scripts/run_all_multitu_projects.sh`

Purpose: End-to-end project runner.

It runs the analyzer on all translation units in a project, builds summary indexes, performs import-aware refinement rounds, and publishes final text and JSON outputs.

Typical usage:

```
PROJECT_PATH=/path/to/project
PROJECT_NAME=my_project
OUTPUT_ROOT=../test-output/real-projects
SUMMARY_IMPORT_ROUNDS=2
../scripts/run_all_multitu_projects.sh
```

Use this script for both curated test projects and real-project pilot runs.

`scripts/build_summary_index.py`

Purpose: Build the project-level summary index.

It consumes a directory of `*.cast-summary.json` files and produces:

```
project-summary-index.json
```

The runner calls this automatically. Manual usage is mainly for debugging:

```
python3 scripts/build_summary_index.py
  test-output/real-projects/my_project/summaries
  --project-kind real
  -o test-output/real-projects/my_project/project-summary-index.json
```

`scripts/validate_classification_labels.py`

Purpose: Validate the stable classification vocabulary and semantic mediation consistency.

Default usage:

```
python3 scripts/validate_classification_labels.py
  --root test-output/real-projects/my_project
```

Strict usage:

```
python3 scripts/validate_classification_labels.py
  --strict-vocabulary
  --root test-output/real-projects/my_project
```

Default mode is recommended for real-project pilot runs. Strict mode is recommended for newly regenerated curated test outputs or when freezing a corpus.

`scripts/validate_cast_summary_schema.py`

Purpose: Validate JSON summaries against the C6f3 schema.

Usage:

```
python3 scripts/validate_cast_summary_schema.py
  --root test-output/real-projects/my_project
```

Without optional `jsonschema`:

```
python3 scripts/validate_cast_summary_schema.py
  --no-jsonschema
  --root test-output/real-projects/my_project
```

This script checks that the stable JSON structure remains suitable for downstream analysis.

`scripts/extract_stable_classifications.py`

Purpose: Export experiment-ready CSV rows from JSON summaries.

Usage:

```
python3 scripts/extract_stable_classifications.py
  --root test-output/real-projects
  --out results/stable-classifications.csv
```

This is the main bridge from tool output to paper tables.

`scripts/validate_cast_text_json_consistency.py`

Purpose: Check that text and JSON outputs agree for curated test cases.

Usage:

```
python3 scripts/validate_cast_text_json_consistency.py
  --root test-output/multitu
```

This is most useful for regression testing. For real projects, use it selectively or on sampled outputs because text output is verbose and primarily intended for debugging.

8. Recommended End-to-End Command Sequence

A typical real-project run should look like this:

```
# 1. Build the tool.
cmake --build build

# 2. Run the project.
cd build

PROJECT_PATH=/path/to/project-or-build-dir
PROJECT_NAME=my_project
OUTPUT_ROOT=../test-output/real-projects
SUMMARY_IMPORT_ROUNDS=2
SOURCE_EXCLUDE_REGEX='/(test|tests|third_party|vendor|build)/'
../scripts/run_all_multitu_projects.sh

cd ..

# 3. Validate classification labels.
python3 scripts/validate_classification_labels.py
  --root test-output/real-projects/my_project

# 4. Validate schema.
python3 scripts/validate_cast_summary_schema.py
  --root test-output/real-projects/my_project

# 5. Export stable classification rows.
mkdir -p results

python3 scripts/extract_stable_classifications.py
```

```
  --root test-output/real-projects/my_project
  --out results/my_project-stable-classifications.csv
```

For a full corpus:

```
python3 scripts/extract_stable_classifications.py
  --root test-output/real-projects
  --out results/all-projects-stable-classifications.csv
```

Then generate aggregate paper tables from the CSV using pandas, R, or another analysis tool.

9. Recommended Interpretation Policy

For paper results, treat the stable JSON classification layer as the authoritative source:

```
stableClassification
stableParamClassification
semanticMediation
paperWildness
```

Use the text output for debugging and examples. Use the JSON summaries and extracted CSV files for quantitative tables.

When default validation passes but strict validation reports metadata-completeness issues, decide whether the missing metadata affects the specific table being extracted. If the classification family, mediation kind, risk, and flags are correct, the result may still be usable for high-level distribution tables. If the table depends on exact field paths, callback targets, or edge labels, then strict-mode issues should be resolved or the affected rows should be excluded from that table with a documented rule.